

Hobby Tracker: Project Abstract

1. Project Overview

This abstract documents the development of a Python based habit tracking application built as the portfolio submission for the course Object-Oriented and Functional Programming with Python. The application enables users to define personal recurring habits, log completed sessions, monitor their progress through a streak system and analyse their habits through a dedicated analytics module. The entire solution is delivered as a command-line interface (CLI) tool written in a single self-contained Python module, with data persisted between sessions in a JSON file.

The project is hosted publicly on GitHub, where all source files, test suite, sample data and documentation can be found: https://github.com/dikshyakhatri8/hobby_tracking_app

2. Technical Approach and Architecture

The application is structured around four classes that each carry a single, clearly defined responsibility, reflecting the object-oriented design principles covered in the course:

- Hobby- encapsulates all data belonging to one habit: title, recurrence cycle, frequency target, description, creation timestamp, log history and streak values. Serialisation to and from a plain dictionary is handled by `to_dict()` and `from_dict()`, keeping the persistence layer fully decoupled.
- HobbyStore: manages all file-system I/O. The class reads and writes the JSON data file and reconstructs Hobby objects on load, returning an empty list gracefully when the file is absent or unreadable.
- HobbyAnalytics, a stateless analytics layer implemented exclusively using functional programming tools: `map()`, `filter()`, `sorted()`, `max()` and `reduce()` together with lambda expressions. It provides functions to list all habits, filter by cycle type, find the longest streak across all habits, compute the mean streak and rank habits by best streak.
- HobbyTracker: the CLI controller that drives a four-option main menu and delegates to the other three classes. It handles all user input, table rendering and the session-logging flow.

The streak calculation is intentionally placed outside any class as a set of pure module-level helper functions. A generator (`_window_sequence`) produces consecutive cycle-window pairs from the earliest log entry to today. For each window, the entry count is compared to the frequency target: a fully closed past window that falls short resets the streak, while the still-open current window is never penalized. This design made the logic straightforward to reason about and to test in isolation.

3. What Went Well

Separating the analytics layer into its own class with a strict functional-only constraint turned out to be a strong design decision. Because HobbyAnalytics never mutates state and every method is expressed as a data transformation, the functions are short, easy to read and trivial to test with exact numeric assertions. The constraint also made it natural to use `reduce()` for aggregation rather than reaching for an imperative loop.

The unit test suite reached 46 tests covering all four classes and every module-level helper. Writing the tests alongside the implementation rather than after it helped catch subtle edge cases in the streak engine early; for example, the distinction between a window that is still open today and one that closed yesterday required a specific guard that would not have been obvious without a failing test to expose it.

The confirmation preview box shown before a new habit is saved was a small but effective usability addition. It reduces the risk of a user committing a typo in the habit name or an incorrect frequency and it fits naturally into the wizard flow without adding any code complexity.

4. Challenges and Pitfalls

The most technically demanding part of the project was designing a streak engine that handles all four cycle types; daily, weekly, monthly and yearly, within a single unified algorithm. The key insight was that every cycle type can be expressed as a sequence of (open-date, close-date) window pairs and the same counting and reset logic applies to all of them. Getting the open-window guard right; ensuring that a habit logged on Monday is not penalised if the rest of the week has not yet passed; required careful reasoning about what 'today' means in relation to each window boundary.

Another pitfall encountered during testing was the use of hardcoded future dates in the streak test cases. Tests written with fixed calendar dates pass when first run but silently fail once those dates move into the past because the streak engine correctly identifies the intervening days as missed windows and resets the count. The fix was to replace all hardcoded dates with expressions relative to `datetime.now().date()`, making the tests permanently date-independent.

5. Features Most Proud Of

The ASCII progress bar is the feature that adds the most visible value to the user experience. A single row of filled (■) and empty (░) blocks immediately communicates how close the current streak is to the personal best, making progress feel tangible without requiring any graphical interface. The same bar appears both when a session is logged and in the full Progress & Stats screen, giving the user consistent feedback throughout the application.

The design of the predefined dataset in hobbies.json is also something worth highlighting. The five habits span three different cycle types (daily, weekly, monthly), include realistic log gaps to demonstrate streak resets, and cover a full four-week period as required by the acceptance criteria. A new user can launch the application and immediately see a meaningful analytics screen rather than an empty state, which helps in evaluating and demonstrating the tool.

6. Conclusion

The finished application meets all acceptance criteria set out in the portfolio brief; it tracks habits across four cycle types, persists data in JSON, exposes a clean CLI, provides a functional analytics module, ships with five predefined habits and four weeks of sample data and is covered by a comprehensive unit test suite. The clear separation between the object-oriented data and persistence layer and the functional analytics layer reflects the dual-paradigm requirement of the course and results in code that is both readable and straightforward to extend.

The complete source code, tests, sample data and documentation are available at:
https://github.com/dikshyakhatri8/hobby_tracking_app